

8주차 계획

목요일까지

call out에 있는 거 각자 하기 + 네트워크와 겹치지 않는 a, b, c, d도 최대한 해결해보기. (2번 노란색 하이라이트 표시된 곳 참고)



- 실제 로컬에서 환경 구축 및 실행 → apk 넣어서 오류 발생 확인 및 오류 해결
 - 네트워크 분석 오류 (목요일까지 늘어질 수 있을 듯)

역할	산출물	병합
A 유예원	app.py / common.py / pipeline_bridge.py / main.py on_progress	PR #6
B (본인)	views/static_view.py + static_data.py + 테스트 51개	PR #7
C 김은아	views/dynamic_view.py, decompiler.py Windows .bat 수정	9982426, c20acf3
D 백소정	src/risk_aggregator.py, views/network_view.py / risk_view.py, 통일 팔레트	PR #8

목요일까지가 1차. 만약에 이때 다 해결이 안되거나 여건상 마무리 못했을 경우 금요일 저녁까지 2차 수정.

▼ 추가 의논하고 싶은 부분(26-08-13 15:31 추가)

지금 결과만 놓고 보면 "이 앱이 악성이다"라고 판단한 것이 아니야. 오히려 현재 대시보드에서 가장 큰 문제는 분석 성공 여부와 앱의 보안 판정이 같은 **정상/실패** 표현으로 섞여 있다는 것 같아.

내가 말한 대시보드 목적이

정상/악성 APK 정적·동적·네트워크 분석 통합 대시보드

라면, UI의 개념을 아예 두 층으로 분리하는 게 좋아.

1. 지금 화면의 **정상** 은 사실상 "분석 성공"

현재:

모듈	화면 표시
정적	✅ 정상
동적	✅ 정상
네트워크	✅ 정상

이건 **APK가 안전하다는 의미로 읽히면 안 돼.**

지금까지 우리가 확인한 흐름을 보면 이 값은 사실:

"정적 분석이 정상적으로 실행됨"

"동적 분석이 정상적으로 실행됨"

"네트워크 분석이 정상적으로 실행됨"

에 가까워.

따라서 여기에는 네가 아까 말했던 것처럼

정상 → 분석 성공

으로 바꾸는 게 맞아.

2. 그럼 네가 분석한 APK는 위험한 건가?

여기서는 "위험 신호가 관찰됐다"와 "악성 APK다"를 구분해야 해.

네 결과를 보면:

원본 이벤트	74
필터링 후	35
평균 후보	35
분석 시간	15.000s

그리고 네트워크에서:

의심 항목	4건
└─ 의심 도메인	1
└─ 의심 IP	3
관측 도메인	4
화이트리스트 통과	3
의심 도메인	1
DNS / TLS	12 / 4

이건 "분석 과정에서 보안상 주의할 만한 요소가 관찰되었다"고 해석하는 게 적절해.

하지만 이것만으로

- ✗ "악성코드가 들어 있다."
- ✗ "악성 APK다."

라고 결론내릴 수는 없어.

특히 네가 이번에 사용한 APK가 **취약점 분석 실습용 APK**라면 더 그래.

예를 들어 실습용 취약 APK는 일부러:

- 평균으로 민감한 데이터를 다루거나
- 취약한 네트워크 통신을 사용하거나
- 테스트용 서버와 통신하거나
- 보안상 좋지 않은 문자열/URL을 포함하거나
- 취약한 인증 로직을 가지고 있을 수 있어.

그러니까 네 결과의 **평균 후보 35** 나 **의심 도메인/IP 4건**은 "악성 행위의 증거"라기보다는 분석 시스템이 발견한 위험 지표(indicator)로 보는 게 정확해.

3. 그래서 대시보드의 구조를 이렇게 바꾸는 걸 추천해

나는 지금 프로젝트 목적이라면 **상단에 최종 판정**을 하나 크게 두는 게 가장 좋다고 생각해.

예를 들어:

APK 보안 분석 결과

△ 의심 (SUSPICIOUS)	
종합 위험도: 68 / 100	
정적 분석	분석 성공
동적 분석	분석 성공
네트워크	분석 성공

그리고 아래에서:

정적 분석

분석 상태	✓ 분석 성공
위험 지표	△ ...

동적 분석

분석 상태	✓ 분석 성공
관찰 이벤트	74
필터링 이벤트	35
평문 후보	35

네트워크 분석

분석 상태	✓ 분석 성공
관측 도메인	4
화이트리스트 통과	3
의심 항목	4
△ 의심 도메인 1건	
△ 의심 IP 3건	

이렇게 "분석이 성공했는가?"와 "무엇이 발견됐는가?"를 분리하는 거야.

4. 그리고 종합 위험도는 별도로 판단

여기가 너희 프로젝트에서 정말 중요해.

현재:

모듈별 기여도
네트워크 40
동적 30
정적 30

이라고 되어 있잖아.

이건 **가중치/기여도**이고, 이것 자체가 앱의 악성 여부를 의미하지는 않아.

최종적으로는 예를 들어:

정적 위험도	55
동적 위험도	72
네트워크 위험도	81

종합 위험도 68

같은 식으로 계산한 다음,

0-29 ● 정상
30-59 ● 주의
60-79 ● 의심
80-100 ● 고위험

처럼 별도의 판정 기준을 두는 거야.

물론 정확한 구간은 너희가 구현한 risk scoring 규칙에 맞춰 정해야 하고.

5. 특히 "악성 APK"라는 표현은 조금 조심하는 게 좋아

프로젝트 설명에서

정상/악성 APK 정적·동적·네트워크 분석 통합 대시보드

라고 하는 건 괜찮아. 프로젝트의 목표를 설명하는 표현이니까.

하지만 실제 결과 화면에서:

● 악성 APK

라고 표시하려면 단순히 위험 점수가 높다는 것 이상의 판정 기준이 필요해.

그래서 나는 결과를:

정상 / 주의 / 의심 / 악성

정도로 나누되, **악성**은 여러 강한 지표가 충족됐을 때만 나오도록 하는 걸 추천해.

그리고 가능하면:

"본 결과는 정적·동적·네트워크 분석에서 관찰된 지표를 기반으로 한 위험도 평가이며, 악성 여부를 확정하지 않습니다."

같은 문구도 작게 넣으면 좋아.

그리고 네가 발견한 "취약 APK인데 정상처럼 보이는" 문제는 오히려 중요한 테스트 포인트야

이번 APK를 넣었는데:

정적 분석 성공
동적 분석 성공
네트워크 분석 성공

이라고 나오면서 내부에는

평문 후보 35
의심 네트워크 4건

이 나오는 상황이지 않아.

이건 **현재 UI가 "분석 성공"을 "안전"으로 잘못 보여주고 있다는 증거야.**

따라서 지금 view를 수정할 때는 단순히 색이나 문구만 고치는 것보다,

분석 상태(Status) ≠ 보안 판정(Verdict)

이 구조를 명확하게 잡는 게 제일 중요해.

내가 추천하는 최종 구조

APK 보안 분석 결과

▲ 의심 APK
종합 위험도 68 / 100

분석 상태

정적 분석

동적 분석

네트워크 분석

✓ 분석 성공

✓ 분석 성공

✓ 분석 성공

위험 지표

정적 분석

동적 분석

네트워크

위험 지표 00건

평균 후보 35건

의심 도메인 1건
의심 IP 3건

모듈별 위험도

정적

동적

네트워크

XX / 100

XX / 100

XX / 100

▲ 주의

본 결과는 분석 과정에서 관찰된 보안 위험 지표를 기반으로
산출된 위험도이며, 악성 여부를 단독으로 확정하지 않습니다.

이렇게 가면 **"정상 APK와 악성 APK를 분석해서 위험도를 보여주는 통합 대시보드"**라는 프로젝트 목적이 훨씬 명확해져.

그리고 지금 네가 가진 **실습용 취약 APK**를 넣은 결과도 **아주 좋은 테스트 케이스**야. "취약하지만 반드시 악성인 것은 아닌 APK"를 넣었을 때 대시보드가 어떻게 표현되는지 확인할 수 있으니까.

최종 목표는 일요일(월요일 오전)까지 테스트 결과 정리하기.

일요일까지

여러 apk 파일로 실험. (anubis.apk 말고도 다른 악성코드앱 있으면 좋고요, 정상적인 앱들도 잘 판별되는지)

- 한 사람당 2개씩 (안 겹치게, 정상 하나, 악성 하나 하면 좋을 듯)
- 다운 받는 곳은 찾아서 남기겠음.
 - 샘플 APK 구하는 경로
 1. 악성 APK: MalwareBazaar(악성 파일 데이터베이스)

- a. 주의! **ELF**는 Linux/Unix 계열의 실행 파일 형식이고, 그중에서도 Mirai는 **IoT/Linux 계열 악성코드**로 유명한 계열이라 'apk' 검색하고 다운 받을 것.
2. 정상 사이트에서 무료 안드로이드 앱 다운로드. (APKPure, APIMirror)
3. 에뮬레이터에 이미 깔려있는 시스템 앱을 뽑아도 됨(adb, emulator 실행 중일 때) :

```
adb shell pm list packages
adb shell pm path <com.android.chrome> # 리스트에서 찾은 앱 중 하나 넣으면 됨
adb pull /data/app/.../base.apk sample.apk # 방금 찾은 경로 복붙
```

예시)

```
package:com.google.android.apps.restore
PS C:\Users\User> adb shell pm path com.google.android.apps.restore
package:/product/priv-app/GoogleRestorePrebuilt/GoogleRestorePrebuilt.apk
PS C:\Users\User> adb pull /product/priv-app/GoogleRestorePrebuilt
/GoogleRestorePrebuilt.apk
/product/priv-app/GoogleRestore... MB/s (5108206 bytes in 0.331s)
```

▼ 에뮬레이터 백그라운드에서 실행 방법

```
adb version
emulator -list-avds # 에뮬레이터 목록

# 에뮬레이터 띄우는 법 2가지
emulator -avd 에뮬레이터이름 # 창 0
emulator -avd 에뮬레이터이름 -no-window
# 창 x

# 다른 cmd/터미널 창에서 켜졌는지 확인
adb devices

# 어떤 AVD가 5554에서 돌고 있는지 확인
adb -s emulator-5554 emu avd name
# 포트 번호는 본인 실행 환경 생각해서 변경하기

# 사용자가 설치한 앱만 보여줌
adb shell pm list packages -3
```

각자 테스트 할 apk 파일(검치면 안됨!!)(민첩해야함...)

이름	정상 파일	악성 파일
백소정		
김은아	Last Day Defense.apk (취약 파일)	f19df82abf0d66389467746f519557e81ca5b373a97e23da7e414204e4931242.apk (malware_test.apk)
유예원		
왕은서		